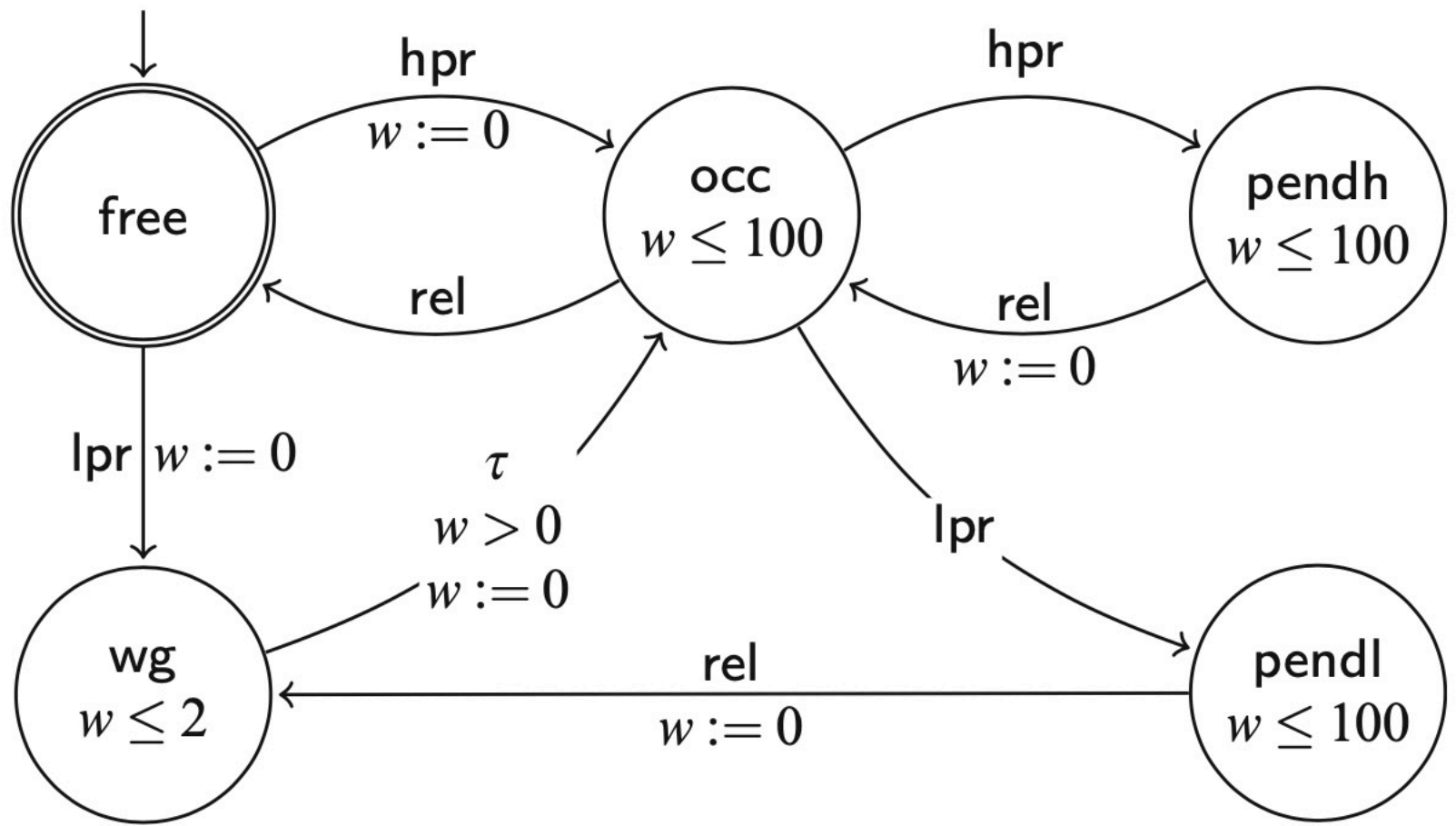


Fig. 7.31 A resource manager modeled through a timed automaton



This diagram represents a **Timed Automaton**, which is essentially a Finite State Automaton (FSA) that has been given a "stopwatch" to track the passage of time.

In your previous models (FSA, PDA), transitions happen instantly based on input. Here, transitions and states are constrained by **time**. The variable w is a **clock** that starts at 0 and increases at a constant rate.

1. The Key "Timed" Components

To read this, you need to understand three specific additions to a standard FSA:

- **Clock Reset** ($w := 0$): When you take this path, you "click" the stopwatch back to zero.
 - **State Invariant** ($w \leq 100$): You are **only** allowed to stay in this circle as long as the clock w is less than or equal to 100. If time hits 101, you *must* have left already.
 - **Guard** ($w > 0$): A condition that must be true for the transition to "fire." You can't take the path unless the clock meets the requirement.
-

2. The Narrative: High vs. Low Priority

This automaton manages a resource (like a printer or a CPU) with two types of requests: **hpr** (High Priority Request) and **lpr** (Low Priority Request).

Phase A: The High Priority Path (Top Row)

1. **free**: The starting point.
2. **hpr** $\rightarrow$ **occ**: A high-priority user grabs the resource. We reset the clock ($w := 0$).
3. **occ** ($w \leq 100$): The resource is "occupied." The user has a maximum of 100 time units to use it. They can finish early via **rel** (release) and go back to **free**.
4. **pendh**: If another high-priority request comes in while the resource is occupied, it becomes "pending high."

Phase B: The Low Priority Path (Bottom Row)

1. **lpr** → **wg**: A low-priority user requests the resource.
2. **wg** ($w \leq 2$): This is a "waiting" state. Notice the invariant is very small ($w \leq 2$). This forces the system to make a decision quickly.
3. τ **transition**: This is an internal/automatic step. If the clock is $w > 0$, the system moves the low-priority user to **occ** (occupied).

Phase C: The Conflict (Preemption)

Look at the transition from **occ** to **pendl**:

- If the resource is currently **occ** (occupied by a high-priority user) and a low-priority request (**lpr**) arrives, it is immediately shoved into **pendl** (pending low).
 - It stays there until the current user releases (**rel**) the resource, at which point it loops back to the waiting phase (**wg**).
-